CHAPTER 6 HOMEWORK

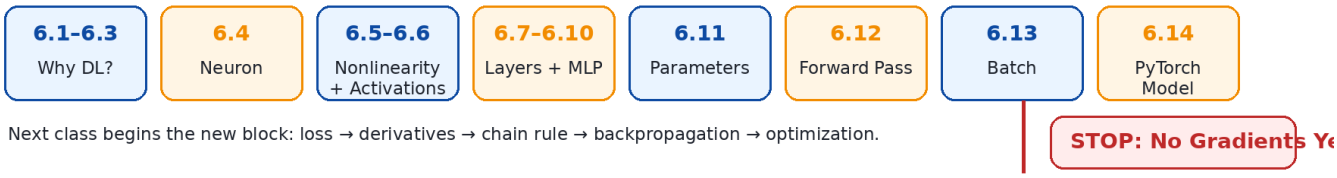
Deep Learning Foundations: Pre-Gradient Assignment

Scope: Sections 6.1–6.14 only | Printed pp. 536–558 | Total: 100 marks

Student Name	Student ID	Submission Date	Score
			/ 100

Chapter 6 Homework Scope: Foundations Before Gradients

Study and solve only the concepts completed in class: Sections 6.1–6.14 (printed pp. 536–558).



IMPORTANT SCOPE BOUNDARY: We stopped at Section 6.14. This homework intentionally does NOT ask you to use derivatives, gradients, chain rule, backpropagation, loss.backward(), autograd, or optimizer updates. Those belong to the next class.

Submission Instructions

- Create one Colab notebook named: QS_K10_C6_PreGradient_HW_<YourName>.ipynb.
- For every mathematical question, show your manual calculation or written derivation first. Then run the instructor-provided Python check cell and attach the output below your answer.
- Code output alone is not a complete mathematical solution. Full marks require the equation, substitution, intermediate steps, and interpretation.
- You may use NumPy and PyTorch only for verification. Do not use gradient-related APIs in this homework.
- Submit: (1) your completed written homework/PDF and (2) your Colab notebook or shareable Colab link, following the class submission channel.

Study Map Before You Start

Section	Book Topic	Printed Page(s)
6.1	Business Problem: Complexity Must Earn Its Place	536–537
6.2	This Week You Will Build	538
6.3	What Changes from Machine Learning to Deep Learning?	539–540
6.4	The Artificial Neuron	541–542
6.5	Why We Need Nonlinearity	542–543
6.6	Activation Functions	543–546
6.7	From One Neuron to One Layer	546–548
6.8	The Multilayer Perceptron	548

Section	Book Topic	Printed Page(s)
6.9	What Does a Hidden Representation Mean?	548–550
6.10	Depth: Why Stack Layers?	550–551
6.11	Parameters: What Does the Network Actually Learn?	551–553
6.12	The Forward Pass	553–554
6.13	From One Example to a Batch	554–556
6.14	A First PyTorch Mental Model	556–558

Q1. Why Deep Learning? Performance vs Complexity

10 marks

Study first: Sections 6.1–6.3 • printed pp. 536–540

A classical baseline achieves F1 = 0.760 with a normalized engineering cost index C = 1.00. Two candidate deep models are tested: Model A has F1 = 0.785 and C = 1.20; Model B has F1 = 0.825 and C = 2.10.

$$\Delta M = M_{DL} - M_{baseline}$$
$$\Delta C = C_{DL} - C_{baseline}$$

1. For Model A, calculate ΔM and ΔC. (2 marks)
2. For Model B, calculate ΔM and ΔC. (2 marks)
3. Calculate the relative F1 improvement of Model A and Model B over the baseline, in percent. (3 marks)
4. In 3–4 sentences, explain which model you would investigate first and why. Your answer must explicitly discuss performance gain and added complexity. (3 marks)

Instructor-provided Colab check cell

```
baseline_f1 = 0.760
baseline_cost = 1.00

models = {
   "A": {"f1": 0.785, "cost": 1.20},
   "B": {"f1": 0.825, "cost": 2.10},
}

for name, m in models.items():
   delta_m = m["f1"] - baseline_f1
   delta_c = m["cost"] - baseline_cost
   relative = delta_m / baseline_f1 * 100
   print(name, "ΔM=", round(delta_m, 4),
       "ΔC=", round(delta_c, 4),
       "relative F1 gain=", round(relative, 2), "%")
```

A:
m = 0.785 - 0.76 = 0.025
c = 1.2 - 1 = 0.2
F1 = 0.025/0.76*100% = 3.29%
B:
m = 0.065
c = 1.1
F1 = 8.55%

A yields modest F1 improvement with little addition to cost. B yields high F1 improvement but C doubles C baseline -> A first if prioritize resource efficiency. B first if error reduction outweights expenses

Q2. Artificial Neuron: Weighted Sum + Activation

15 marks

Study first: Section 6.4 and ReLU/Sigmoid from 6.6 • printed pp. 541–546

Consider one neuron with the following input, weights, and bias:

$$x = [0.6, -0.2, 0.9, 0.4]^T$$
$$w = [1.1, -0.5, 0.8, -0.3]^T$$
$$b = -0.25$$
$$z = w^T x + b$$

5. Expand the dot product and calculate z by hand. Show every multiplication. (6 marks)
6. Calculate ReLU(z). (2 marks)
7. Calculate sigmoid(z) = 1 / (1 + e^(−z)) to at least 4 decimal places. (3 marks)
8. Explain in one sentence what the weights represent, and in one sentence what the bias does. (2 marks)
9. Has the neuron “learned” anything simply because you completed this forward calculation? Explain. (2 marks)

Instructor-provided Colab check cell

```
import numpy as np

x = np.array([0.6, -0.2, 0.9, 0.4])
w = np.array([1.1, -0.5, 0.8, -0.3])
b = -0.25

z = w @ x + b
relu = max(0.0, z)
sigmoid = 1 / (1 + np.exp(-z))

print("z =", z)
print("ReLU(z) =", relu)
print("sigmoid(z) =", sigmoid)
```

z = 1.1 * 0.6 + (-0.5) * (-0.2) + 0.8 * 0.9 + (-0.3) * 0.4 + (-0.25)
= 0.66 + 0.1 + 0.72 - 0.12 -0.25 = 1.11

reLU (z) = max(0,z) = max (0,1.11) = 1.11

sigmoid(z) - 1/(1+e^(-1.11)) = 0.7521

w quantifes relative importance of each input feature toward neuron's output
b acts as threshold offset, enabling activation function to shift along the axis independently of input values

no, this is forward pass

Q3. Why Nonlinearity? + Activation Functions

15 marks

Study first: Sections 6.5–6.6 • printed pp. 542–546

Part A — Algebraic proof

$$h = W_1x + b_1$$
$$y = W_2h + b_2$$

10. Substitute the first equation into the second and show that the two linear layers can be written as y = W′x + b′. Define W′ and b′. (7 marks)
11. Explain why this means “more linear layers” alone do not give nonlinear modeling power. (2 marks)

Part B — Activation behavior

12. For z ∈ {−3, −1, 0, 1, 3}, compute Sigmoid(z), Tanh(z), and ReLU(z). Present the results in a table. (4 marks)
13. Which activation is the chapter’s practical choice for the first MLP hidden layers, and what is one caution associated with it? (2 marks)

Instructor-provided Colab check cell

```
import numpy as np

z = np.array([-3., -1., 0., 1., 3.])

sigmoid = 1 / (1 + np.exp(-z))
tanh = np.tanh(z)
relu = np.maximum(0, z)

for zi, si, ti, ri in zip(z, sigmoid, tanh, relu):
    print(f"z={zi:>4.1f} | sigmoid={si:.4f} | tanh={ti:.4f} | ReLU={ri:.4f}")
```

y = W2(W1x+b1)+b2 = W2W1x + W2b1 + b2
W' = W2W1, b' = W2b1 + b2

stacking more linear layers is equivalent to a single linear layer with weight W' and bias' b'

Q4. From One Neuron to a Layer + Hidden Representation

15 marks

Study first: Sections 6.7–6.9 • printed pp. 546–550

A layer receives x ∈ R^4 and has 3 hidden units. Use:

$$x = [1.0, 0.5, -1.0, 2.0]^T$$
$$W = [[0.5, 1.0, -0.5, 0.2], [-0.3, 0.8, 0.4, -0.6], [1.2, -0.4, 0.1, 0.5]]$$
$$b = [0.1, -0.2, 0.3]^T$$
$$z = Wx + b, \quad h = \mathrm{ReLU}(z)$$

14. Write the shapes of x, W, b, z, and h. (4 marks)
15. Compute z manually. Show one full row calculation in detail. (5 marks)
16. Apply ReLU to obtain h. (2 marks)
17. Explain why h is called a hidden representation rather than simply another copy of the original input. (4 marks)

Instructor-provided Colab check cell

```
import numpy as np

x = np.array([1.0, 0.5, -1.0, 2.0])
W = np.array([
   [ 0.5,  1.0, -0.5,  0.2],
   [-0.3,  0.8,  0.4, -0.6],
   [ 1.2, -0.4,  0.1,  0.5],
])
b = np.array([0.1, -0.2, 0.3])

z = W @ x + b
h = np.maximum(0, z)

print("x shape:", x.shape)
print("W shape:", W.shape)
print("b shape:", b.shape)
print("z =", z)
print("h = ReLU(z) =", h)
```

h = [max(0,2), max(0,-1.7), max(0,2.2)]^T = [2,0,2.2]^T

x: (4x1)
W: (3x4)
b: (3x1)
z: (3x1)
h: (3x1)

h is constructed by taking linear projections of inputs and applying non linear activations.

Q5. Depth and Parameter Counting — 15 marks

Study first: Sections 6.10–6.11 • printed pp. 550–553

$$\text{Dense layer parameters} = d \times h + h = h(d+1)$$

Network A has architecture 6 → 16 → 8 → 1. Network B has architecture 6 → 64 → 32 → 1.

18. For Network A, calculate the parameters in each layer and the total. (5 marks)
19. For Network B, calculate the parameters in each layer and the total. (5 marks)
20. How many times more trainable parameters does Network B have than Network A? (2 marks)
21. Give three engineering risks/costs that can increase as depth/width/capacity increases. Use the book's discussion of depth and parameters. (3 marks)

Instructor-provided Colab check cell

```
def dense_params(in_dim, out_dim):
    return in_dim * out_dim + out_dim

def mlp_params(sizes):
    per_layer = []
    for a, b in zip(sizes[:-1], sizes[1:]):
        per_layer.append(dense_params(a, b))
    return per_layer, sum(per_layer)

for name, sizes in {
    "Network A": [6, 16, 8, 1],
    "Network B": [6, 64, 32, 1],
}.items():
    layer_counts, total = mlp_params(sizes)
    print(name, sizes)
    print("per layer =", layer_counts)
    print("total =", total)
    print()
```

```
A:
1: 112
2: 136
3: 9
total: 257

B:
1: 448
2: 2080
3: 33
total: 2561
```

2561/257 = 9.965

overfitting, high computational cost, optimization difficulty

Q6. Full Forward Pass Through a Tiny MLP — 15 marks

Study first: Section 6.8 + Section 6.12 • printed p. 548 and pp. 553–554

Perform one complete forward pass through the network below. There is NO training and NO gradient calculation in this question.

$$x = [1.0, -2.0, 0.5]^T$$
$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad h^{(1)} = \text{ReLU}(z^{(1)})$$
$$z^{(2)} = W^{(2)}h^{(1)} + b^{(2)}, \quad h^{(2)} = \text{ReLU}(z^{(2)})$$
$$z^{(3)} = W^{(3)}h^{(2)} + b^{(3)}, \quad p = \text{sigmoid}(z^{(3)})$$

Use the parameters:

$$W^{(1)} = [[0.5, -0.2, 0.4], [-0.3, 0.8, -0.5], [0.7, 0.1, 0.2]]$$
$$b^{(1)} = [0.1, -0.1, 0.0]^T$$
$$W^{(2)} = [[0.6, -0.4, 0.3], [-0.2, 0.5, 0.7]], \quad b^{(2)} = [0.05, -0.15]^T$$
$$W^{(3)} = [[0.8, -0.6]], \quad b^{(3)} = [0.2]$$

22. Calculate z^(1) and h^(1). (5 marks)
23. Calculate z^(2) and h^(2). (4 marks)
24. Calculate the final raw logit z^(3). (3 marks)
25. Convert the logit to p = sigmoid(z^(3)). State clearly: has the network learned anything yet? (3 marks)

Instructor-provided Colab check cell

z^1 = [1.2,-2.25,0.6]^T
h^1 = [1/2,0,0.6]^T
z^2 = [0.95, 0.03]^T

h^2 = [0.95,0.03]^T
z^3 = 0.942
p = 0.7195
no, output is just generated solely from initial parameters without any optimization

```
import numpy as np

x = np.array([1.0, -2.0, 0.5])
W1 = np.array([
   [ 0.5, -0.2,  0.4],
   [-0.3,  0.8, -0.5],
   [ 0.7,  0.1,  0.2],
])
b1 = np.array([0.1, -0.1, 0.0])

W2 = np.array([
   [ 0.6, -0.4, 0.3],
   [-0.2,  0.5, 0.7],
])
b2 = np.array([0.05, -0.15])

W3 = np.array([[0.8, -0.6]])
b3 = np.array([0.2])

relu = lambda v: np.maximum(0, v)

z1 = W1 @ x + b1
h1 = relu(z1)
z2 = W2 @ h1 + b2
h2 = relu(z2)
z3 = W3 @ h2 + b3
p = 1 / (1 + np.exp(-z3))

print("z1 =", z1)
print("h1 =", h1)
print("z2 =", z2)
print("h2 =", h2)
print("raw logit z3 =", z3)
print("sigmoid probability p =", p)
```

Q7. From One Example to a Batch — 10 marks

Study first: Section 6.13 • printed pp. 554–556

$$X \in R^{(B\times d)}, \quad W \in R^{(h\times d)}$$
$$Z = XW^T + 1b^T, \quad Z \in R^{(B\times h)}$$

Let B = 4, d = 3, h = 2 and use:

$$X = [[1,0,2], [0,1,-1], [2,1,0], [-1,2,1]]$$
$$W = [[0.5, -0.2, 0.3], [-0.4, 0.7, 0.1]]$$
$$b = [0.1, -0.2]^T$$

26. Write the shapes of X, W, W^T, b, and Z. (4 marks)
27. Manually calculate the first output row Z[0]. (3 marks)
28. Use the checker code to compute the complete Z matrix. In one sentence, explain what each row of Z represents. (3 marks)

Instructor-provided Colab check cell

each row i of Z represents the pre activation vector for the i-th sample in the mini batch

X:4,3
W:2,3
W^T:3,2
b:2.1
Z:4.2

Z1,1 = 1.2
Z1,2 = -0.4

```
import numpy as np

X = np.array([
   [ 1., 0.,  2.],
   [ 0., 1., -1.],
   [ 2., 1.,  0.],
   [-1., 2.,  1.],
])
W = np.array([
   [ 0.5, -0.2, 0.3],
   [-0.4,  0.7, 0.1],
])
b = np.array([0.1, -0.2])

Z = X @ W.T + b

print("X shape =", X.shape)
print("W shape =", W.shape)
print("W.T shape =", W.T.shape)
print("b shape =", b.shape)
print("Z shape =", Z.shape)
print("Z =\n", Z)
```

Q8. PyTorch Mental Model: Tensor → Module → Forward Pass

5 marks

Study first: Section 6.14 • printed pp. 556–558

Run the code below in Colab. Do not add a loss function, backward pass, or optimizer yet.

Instructor-provided Colab check cell

```
import torch
import torch.nn as nn

x = torch.tensor([
   [0.8, 0.5, 0.2],
   [0.1, 0.4, 0.9],
   [0.6, 0.3, 0.7],
   [0.2, 0.9, 0.1],
], dtype=torch.float32)

class SmallMLP(nn.Module):
   def __init__(self, input_dim):
      super().__init__()
      self.network = nn.Sequential(
         nn.Linear(input_dim, 32),
         nn.ReLU(),
         nn.Linear(32, 8),
         nn.ReLU(),
         nn.Linear(8, 1),
      )

   def forward(self, x):
      return self.network(x)

model = SmallMLP(input_dim=3)
logits = model(x)

print("input shape:", x.shape)
print("output/logit shape:", logits.shape)
print("\nTrainable parameters:")
for name, parameter in model.named_parameters():
   print(name, parameter.shape, "requires_grad=", parameter.requires_grad)
```

in: 4,3
out: 4,1

1: weight 32,3 bias 32
2: weight 8,32 bias 8
3: weight 1,8 bias 1

code only pass the input tensor thorugh model to obtain the logit.
does not perform backpropagation
does not call an optimizer to update the weights

29. Record the input tensor shape and output tensor shape. (1 mark)
30. List the weight and bias tensor shapes printed for all three Linear layers. (2 marks)
31. Explain why this code performs a forward pass but does not train the model. (2 marks)

Final Submission Checklist

- ☐ I solved all mathematical parts before using the Python checker.
- ☐ I included intermediate calculations, not only final numbers.
- ☐ I included Colab output/screenshots for every code-check section.
- ☐ I did not use gradients, backward(), autograd, or optimizer updates.
- ☐ I answered interpretation questions in my own words.
- ☐ I reviewed the specified Chapter 6 section/page before each question.
- ☐ My Colab notebook runs top-to-bottom without errors.

PREP FOR NEXT CLASS

Be comfortable with forward computation, shapes, hidden representations, and parameter counts. Next we will ask how the network knows what to change.